

Rapport de stage

**Méthodes hybrides pour l'ordonnancement sous
contraintes de ressources complexes**

July 17, 2026

Stagiaire :

Alex TSAN alex.tsan@utoulouse.fr

Tuteurs de stage :

Christian ARTIGUES
Carla JUVIN
Pierre LOPEZ

Mot clés : Recherche opérationnelle; Programmation par contrainte; Décomposition de Benders;
Programmation Linéaire

Abstract :

La recherche d'efficacité opérationnelle dans l'industrie moderne nécessite la résolution de problèmes d'ordonnancement et d'allocation de ressources hautement flexibles et fortement contraints. Ce stage a pour objectif d'explorer, d'implémenter et de comparer différents paradigmes d'optimisation exacte pour résoudre ces problèmes complexes. Plus précisément, nous évaluons la performance et le passage à l'échelle d'une formulation monolithique en programmation par contraintes (CP) face à une technique de décomposition hybride. Dans un premier temps, nous développons un modèle de programmation par contraintes où les activités sont découpées en sous-intervalles de durée unitaire afin de capturer la flexibilité d'affectation dynamique via des contraintes alternatives. Nous analysons comment cette formulation monolithique introduit de sévères symétries combinatoires, saturant le moteur d'inférence du solveur sur les instances de grande taille. Pour contourner ces limites, nous concevons et implémentons un framework de Décomposition de Benders Logique (LBBD). Cette

approche sépare le problème : un Problème Maître CP gère les décisions d'ordonnancement globales, tandis qu'un Sous-Problème indépendant valide la faisabilité de l'allocation des compétences via des algorithmes de flot maximum. Les résultats obtenus sur des instances de référence démontrent que le framework hybride LBBD surclasse drastiquement le modèle CP monolithique initial, réduisant les temps de résolution tout en prouvant l'optimalité.

Contents

Introduction	1
0.1 Contexte et Présentation du LAAS-CNRS	1
0.2 Énoncé du problème et objectifs du stage	2
0.3 Annonce du plan	5
1 Programmation linéaire en nombres entiers	6
1.1 Structure mathématique d'un modèle PLNE	6
1.2 Mécanismes de résolution :	6
1.3 Application au problème du MS-RCPSP-AF	7
1.3.1 Ensembles	7
1.3.2 Paramètres	7
1.3.3 Variables de décision	7
1.3.4 Fonction objectif	8
1.3.5 Contraintes	8
1.4 Analyse des limites et verrous calculatoires de la formulation PLNE	9
1.4.1 Explosion dimensionnelle avec l'horizon temporel	9
1.4.2 Faiblesse de la relaxation continue	9
1.4.3 Linéarisation des relations logiques	9
2 Programmation par Contraintes	10
2.1 Formalisme des Problèmes de Satisfaction de Contraintes (CSP)	10
2.2 Mécanismes de résolution : Propagation et Recherche	10
2.3 Primitives d'ordonnancement de CP Optimizer	11
2.3.1 Variables d'intervalles (Interval Variables)	11
2.3.2 Contraintes temporelles	11
2.3.3 Ressources disjonctives et cumulatives	12
2.4 Modélisation CP monolithique pour le MS-RCPSP-AF	12
2.4.1 Ensembles et paramètres	12
2.4.2 Variables de décision	12
2.4.3 Formulation mathématique	12
2.4.4 Explication des contraintes	13

2.4.5	Contraintes redondantes de capacité globale	13
2.5	Contraintes de bris de symétrie	14
2.6	Analyse des limites et verrous combinatoires du modèle CP monolithique	14
2.6.1	Explosion du nombre de variables	14
2.6.2	Symétries massives	15
3	Décomposition de Benders basée sur la Logique	16
3.1	Principes de la Décomposition de Benders basée sur la Logique (LBBD)	16
3.2	Problème Maître d'Ordonnancement	16
3.2.1	Formulation mathématique	17
3.3	Sous-problème d'Affectation	17
3.3.1	Formulation sous forme de Flot Maximum (Max-Flow)	17
3.4	Génération des coupes de Benders	18
3.4.1	Formulation duale et Coupe Minimale (Min-Cut)	18
3.4.2	Formulation de la coupe injectée	19
4	Évaluation Expérimentale	20
4.1	Protocole expérimental et instances de test	20
4.1.1	Environnement de test	20
4.1.2	Description du jeu de données	20
4.2	Comparaison globale des approches	20
4.2.1	Analyse des taux de résolution et des temps de calcul	21
4.2.2	Comparaison directe sur les instances faciles (communes)	21
4.3	Analyse fine des performances et discussion	21
4.3.1	Le phénomène de résolution instantanée par Benders	21
4.3.2	Analyse de l'impact des contraintes de bris de symétrie	21
4.4	Visualisations graphiques	22
	Conclusion et Perspectives	24

Introduction

0.1 Contexte et Présentation du LAAS-CNRS

Ce document s'inscrit dans le cadre du stage de fin d'année de Master 1 Informatique, parcours Intelligence Artificielle Fondements et Applications (IAFA) à l'Université de Toulouse. Ce stage a été réalisé au sein du LAAS-CNRS (Laboratoire d'Analyse et d'Architecture des Systèmes) à Toulouse. Fondé en 1968, le LAAS est une Unité Propre de Recherche (UPR 8001) du Centre National de la Recherche Scientifique (CNRS), rattachée principalement à l'Institut des sciences de l'information et de leurs interactions (IS2I) et à l'Institut des sciences de l'ingénierie et des systèmes (INSIS). Situé à Toulouse au cœur du complexe scientifique de Rangueil, le laboratoire est conventionné avec les principaux établissements d'enseignement supérieur de la région : l'Université de Toulouse, l'INSA Toulouse, l'INP Toulouse et l'ISAE-SUPAERO. Il regroupe aujourd'hui un effectif d'environ 650 personnes, composé de chercheurs, d'enseignants-chercheurs, d'ingénieurs, de techniciens, de doctorants et de post-doctorants.

Les activités de recherche du LAAS-CNRS se structurent autour de quatre champs disciplinaires principaux l'Automatique, l'Informatique, les Micro et Nano Systèmes, et la Robotique, créant ainsi un écosystème scientifique unique qui va de la conception du composant matériel jusqu'aux algorithmes d'optimisation et de décision.

L'activité scientifique du LAAS-CNRS est structurée autour de six départements de recherche complémentaires DO, GE, HOPES, MNBT, RISC et ROB, qui couvrent l'ensemble du spectre des sciences du numérique, des composants physiques aux systèmes autonomes.

- **Décisions et Optimisation** : Le département DO développe des théories mathématiques et des outils algorithmiques pour la prise de décision et la commande de systèmes complexes. Ses recherches s'articulent autour de deux approches complémentaires : d'une part, la synthèse de lois de décision via des techniques d'optimisation (souvent appliquées à des signaux continus) et, d'autre part, la résolution de problèmes d'optimisation combinatoire purement discrets. Faisant face à la complication de données hétérogènes (combinant continu et discret), ses travaux trouvent des applications majeures dans les réseaux électriques, les transports et l'industrie du futur. C'est dans ce cadre, axé sur l'optimisation combinatoire sous contraintes, que s'intègrent les recherches de l'équipe ROC en ordonnancement et planification.
- **Gestion de l'Energie** : Le département GE se consacre à l'optimisation de l'efficacité énergétique, à la robustesse et à la fiabilité des architectures de conversion de puissance. Pour répondre aux défis de la transition écologique, de la décentralisation des énergies renouvelables et de l'électrification des transports, il conçoit des composants électroniques miniatures et des systèmes de gestion intelligente de l'énergie. Ses méthodologies s'appuient sur la modélisation multi-physique, l'intégration de semi-conducteurs à large bande interdite (GaN, SiC, diamant) et l'utilisation de la salle blanche du laboratoire. Le département valide notamment ses prototypes au sein du bâtiment expérimental connecté ADREAM.
- **Systèmes Hyperfréquences et Photoniques** Le département HOPES explore la convergence entre l'électronique de pointe et la photonique pour concevoir les briques de base de l'Internet des objets (IoT) et des futurs systèmes cyber-physiques. Ses recherches visent à créer des composants miniatures, multifonctionnels et à très faible consommation d'énergie, capables de communiquer et d'interagir intelligemment avec leur environnement. En combinant la physique des matériaux, l'ingénierie optique et les technologies hyperfréquences, le département développe des capteurs, des dispositifs d'imagerie et des solutions de communication sans fil innovantes pour l'espace, la santé et les systèmes autonomes.

- **Micro Nano Bio Technologies** : Le département MNBT combine des approches fondamentales et appliquées pour concevoir des nanomatériaux (semi-conducteurs, organiques ou vivants) et des nano-objets, puis les intégrer dans des dispositifs technologiques complexes. Ses activités ciblent trois grands défis sociétaux : les technologies pour le numérique (nanoélectronique, spintronique, quantique), les technologies pour la santé (microfluidique, organes sur puce, dispositifs biomédicaux) et la surveillance environnementale (capteurs de qualité de l'eau et de l'air). Le département inscrit pleinement ses pratiques de fabrication dans une démarche d'éco-conception et de développement durable.
- **Réseaux Informatique Systèmes de Confiance** : Le département RISC cible la maîtrise des architectures informatiques et des réseaux de communication complexes, ubiquitaires et distribués. Ses recherches se focalisent sur la cybersécurité (logicielle, matérielle et réseaux), l'orchestration dynamique des ressources cloud (le Cloud Continuum) et le développement d'une intelligence artificielle digne de confiance et frugale en énergie. Axé sur l'analyse et l'ingénierie des systèmes critiques et des algorithmes répartis, le département fournit des solutions logicielles et des modèles stochastiques pour garantir des propriétés strictes de sûreté de fonctionnement, de tolérance aux fautes et de respect des contraintes temps réel.
- **Robotique** : Le département ROB déploie une recherche pluri-thématique dédiée à l'autonomie motrice et cognitive des systèmes physiques opérant dans des environnements réels et non maîtrisés. Ses domaines d'expertise couvrent la planification et la génération de mouvements pour les structures anthropomorphes ou instables, le couplage temps réel entre la perception sensorielle (vision, audition via fusion probabiliste) et l'action, ainsi que la gestion des interactions cognitives et physiques entre le robot, l'Homme et son milieu. Très ancré dans la philosophie de la science ouverte, le département se distingue par la conception et la distribution en libre accès de nombreux logiciels de référence de la communauté internationale.

Ce stage a été effectué au sein du département Décisions et Optimisation (DO), au sein de l'équipe de recherche ROC (Recherche Opérationnelle, Combinatoire et Contraintes) dirigée par Christian Artigues. L'équipe ROC se consacre à la résolution de problèmes d'optimisation combinatoire discrets et complexes, souvent issus du monde industriel. Son objectif est de concevoir des modèles mathématiques et des algorithmes performants pour trouver les meilleures décisions possibles face à un nombre gigantesque de choix. Ses recherches se déploient principalement autour de trois grands piliers méthodologiques :

- **La Programmation Linéaire** : Formulations mathématiques rigoureuses et approches polyédrales.
- **La Programmation par Contraintes** : Exploitation de structures logiques, temporelles (variables d'intervalles) et de propagation.
- **Les Méthodes Hybrides et Décompositions** : Couplage de plusieurs paradigmes (comme la décomposition de Benders logique ou le *Branch-and-Price*) pour découper et vaincre les ve rrous calculatoires des problèmes de grande taille.

0.2 Énoncé du problème et objectifs du stage

Le problème abordé au cours de ce stage s'inscrit dans le domaine de l'optimisation combinatoire et de l'ordonnancement sous contraintes de ressources. Plus précisément, il s'agit du problème d'ordonnancement de projet sous contraintes de ressources multi-compétences avec flexibilité d'affectation (Multi-Skill Resource-Constrained Project Scheduling Problem with Allocation Flexibility, ou MS-RCPSP-AF).

Dans sa formulation classique (RCPSP), l'ordonnancement consiste à planifier un ensemble de tâches liées par des contraintes de précédence, tout en respectant la capacité limitée de ressources cumulatives c'est-à-dire des ressources disponibles en quantité globale constante à chaque instant et pouvant être consommées simultanément par plusieurs tâches distinctes (comme une puissance électrique maximale, un parc de machines identiques ou un budget par période), avec pour objectif de minimiser la durée totale du projet (*makespan*). La variante MS-RCPSP (MS pour *Multi-Skill*) intègre la gestion de ressources humaines caractérisées par des profils hétérogènes : les opérateurs possèdent une ou plusieurs compétences spécifiques (la soudure ou la plomberie, par exemple). Les besoins d'une tâche ne sont alors plus définis par une quantité brute de main-d'œuvre, mais par des exigences technologiques précises en compétences ; c'est-à-dire qu'une tâche peut, par exemple, exiger un électricien et un soudeur. Dans ce cadre traditionnel, l'affectation est dite rigide, ce qui implique que les opérateurs sélectionnés sont immobilisés sur une tâche, de son déclenchement jusqu'à sa finalisation.

L'introduction de la flexibilité d'affectation (MS-RCPSP-AF) lève cette hypothèse restrictive en autorisant la rotation, le remplacement ou la réaffectation dynamique des opérateurs au cours de l'exécution d'une même activité. La seule condition requise est que le profil d'exigences en compétences de la tâche soit intégralement couvert à chaque unité de temps de sa durée d'exécution.

Pour illustrer concrètement l'intérêt de cette flexibilité d'allocation (AF), la Figure 1 compare deux scénarios d'exécution d'un projet composé de trois tâches (*Tâche A* de durée 10 requérant la compétence s_1 , *Tâche B* de durée 5 requérant s_2 , et *Tâche C* de durée 5 requérant s_3) à réaliser par deux travailleurs (W_1 maîtrisant $\{s_1, s_2\}$ et W_2 maîtrisant $\{s_1, s_3\}$) :

- **Scénario 1 (Sans Flexibilité) :** L'affectation est rigide. Si le travailleur W_1 commence la Tâche B à l'instant 0, il y reste affecté jusqu'à sa fin à l'instant 5. La Tâche A (exigeant s_1) ne peut pas démarrer en parallèle car l'unique autre travailleur disponible, W_2 , ne peut pas être affecté à la Tâche A du début à la fin sous peine de ne pas pouvoir réaliser la Tâche C (exigeant la compétence exclusive s_3 de W_2) à l'instant 5. Par conséquent, la Tâche A est décalée et ne commence qu'à l'instant 5, affectée à W_1 , ce qui donne un Makespan total de 15 heures.
- **Scénario 2 (Avec Flexibilité) :** Un passage de relais dynamique est possible. La Tâche A débute dès l'instant 0, prise en charge par W_2 pour sa première moitié. À l'instant 5, la Tâche C doit démarrer et requiert W_2 pour sa compétence s_3 . Grâce à la flexibilité, W_2 cède sa place sur la Tâche A à W_1 (qui vient de terminer la Tâche B). La Tâche A se poursuit sans interruption et se termine à l'instant 10, tout comme la Tâche C. Le Makespan est ainsi réduit de 15h à 10h.

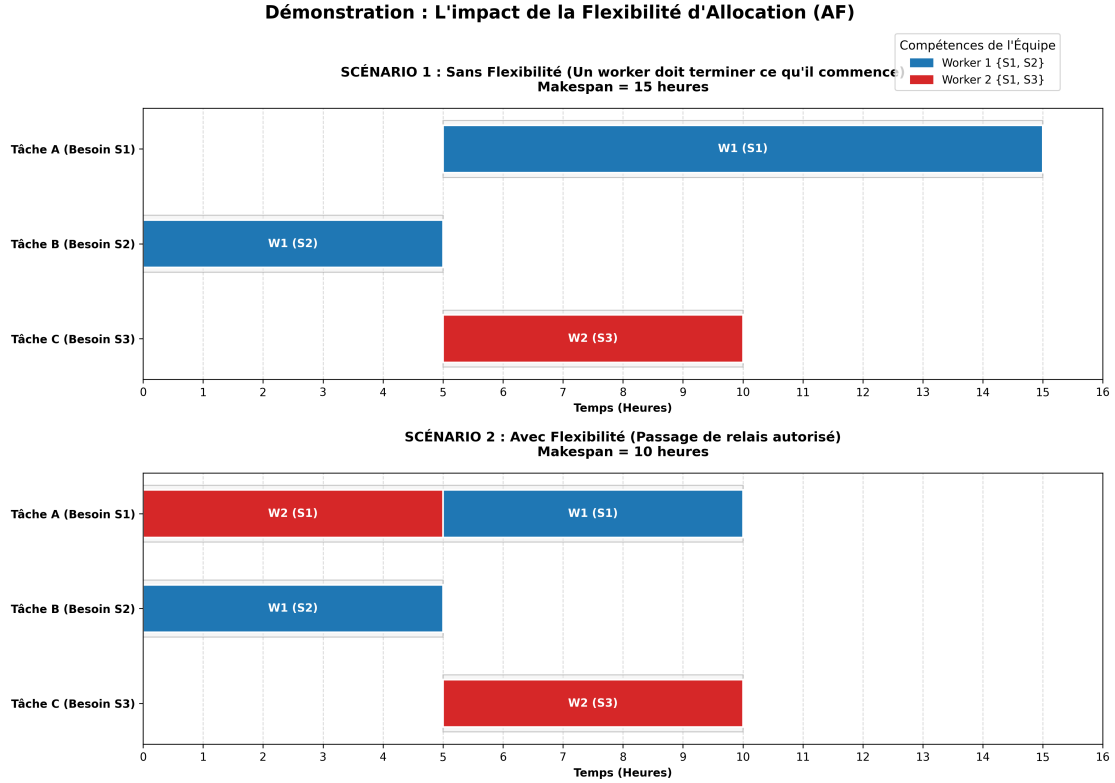


Figure 1: Illustration de l'impact de la flexibilité d'affectation (AF) sur le Makespan d'un projet

Si cette flexibilité offre un avantage industriel évident en optimisant l'usage de la main-d'œuvre et en réduisant le makespan, elle induit une complexité mathématique critique. Contrairement aux approches classiques où la décision d'affectation est prise globalement pour la durée de l'activité, la flexibilité impose de valider la faisabilité de l'affectation de manière discrétisée dans le temps. Il en résulte une explosion combinatoire massive, accentuée par de fortes symétries lorsque plusieurs opérateurs partagent des compétences identiques. Ces propriétés satureront rapidement les mécanismes d'élagage des solveurs standards, constituant un verrou calculatoire majeur pour les méthodes exactes.

La littérature scientifique de la recherche opérationnelle s'est largement penchée sur le problème classique d'ordonnancement de projets (RCPSp). L'intégration de ressources multi-compétences, introduite par Bellenguez-Morineau et Néron [1, 2], a suscité un intérêt croissant ces dernières années, comme en témoigne la revue de littérature récente de Snauwaert et Vanhoucke [3]. Les approches exactes traditionnelles reposent souvent sur la programmation linéaire en nombres entiers (MILP), étudiée et comparée notamment par Almeida et al. [4], mais ces formulations souffrent d'un mauvais passage à l'échelle lorsque l'horizon temporel s'allonge. Pour y remédier, des approches basées sur la génération de colonnes (*Branch-and-Price*) ont été explorées [5, 6]. Parallèlement, la Programmation par Contraintes (PPC) s'est imposée comme une alternative puissante pour modéliser des contraintes complexes et de la préemption [7, 8]. Néanmoins, ces modèles monolithiques atteignent rapidement leurs limites sur des instances de taille industrielle en raison de l'explosion combinatoire induite par la flexibilité d'affectation dynamique (MS-RCPSp-AF). C'est pourquoi la recherche s'oriente vers des méthodes de décomposition hybrides. La Décomposition de Benders basée sur la Logique (LBBD), introduite formellement par Hooker [9, 10] et analysée sur le plan computationnel par Cire et al. [11], apparaît comme l'un des frameworks les plus prometteurs de l'état de l'art. Contrairement aux schémas de Benders classiques qui délèguent l'ordonnancement au sous-problème, les approches d'ordonnancement multi-compétences peuvent inverser ce schéma en résolvant l'ordonnancement global dans le problème maître en CP et l'affectation dans le sous-

problème. Cette inversion, introduite par Yi-fan et al. [12] pour le MSPSP classique, a été étendue par Juvin et al. [13] pour modéliser la flexibilité d'affectation (*Allocation Flexibility* ou AF) dans le cadre du MS-RCPSP-AF, permettant ainsi le remplacement dynamique des travailleurs au cours de l'exécution des tâches.

0.3 Annonce du plan

Afin de rendre compte de la démarche scientifique adoptée, des différents paradigmes modélisés, des méthodes de résolution implémentées, et de l'analyse comparative qui en découle, ce manuscrit est structuré en quatre chapitres principaux :

Le Chapitre 1 introduit la modélisation mathématique initiale du problème sous la forme d'un programme linéaire en nombres entiers (ILP). Cette première approche à temps discret permet de formaliser rigoureusement les contraintes d'affectation dynamique et de précedence, tout en exposant les méthodes de linéarisation utilisées et en mettant en évidence les verrous calculatoires inhérents à ce type de formulation.

Le Chapitre 2 explore une approche monolithique alternative fondée sur la Programmation par Contraintes (CP). Nous y décrivons les méthodes de propagation et de filtrage sous-jacentes, la modélisation et l'implémentation du problème s'appuyant sur des structures de sous-intervalles unitaires, et analysons l'impact de la réification logique indispensable pour capturer la flexibilité d'affectation sur les performances et la saturation du solveur face à l'extension de l'horizon temporel.

Le Chapitre 3 présente le framework de Décomposition de Benders basé sur la Logique (LBBD), développé pour s'affranchir des limites des modèles monolithiques précédents. Ce chapitre détaille les méthodes de décomposition et de génération de coupes logiques employées, ainsi que l'articulation algorithmique entre le Problème Maître (CP), qui orchestre l'ordonnancement global en temps continu, et le Sous-Problème d'affectation spatiale, résolu efficacement à l'aide d'algorithmes de flot maximum issus de la théorie des graphes.

Enfin, le Chapitre 4 est dédié à l'évaluation expérimentale et à l'analyse critique des performances des trois approches sur des instances de référence de la littérature. Cette confrontation empirique met en lumière les capacités de passage à l'échelle, la pertinence des choix méthodologiques et algorithmiques, ainsi que l'efficacité de chaque paradigme face à l'explosion combinatoire induite par la flexibilité.

1 Programmation linéaire en nombres entiers

La Programmation Linéaire en Nombres Entiers (PLNE, ou ILP pour *Integer Linear Programming* en anglais) est un paradigme mathématique d'optimisation exacte largement répandu en Recherche Opérationnelle pour résoudre des problèmes de décision complexes. Depuis les travaux pionniers du milieu du XXe siècle, la PLNE s'est imposée comme un outil de référence pour modéliser des problèmes NP-difficiles [14], combinant une expressivité mathématique rigoureuse avec la puissance de solveurs numériques modernes [15].

Dans le cadre de l'ordonnancement de projets sous contraintes de ressources (RCPSP) et de ses extensions multi-compétences (MS-RCPSP), la PLNE a historiquement constitué la première approche de résolution exacte [16]. Les formulations linéaires permettent de décrire précisément les relations temporelles entre tâches et les contraintes d'affectation de ressources. En particulier, les formulations dites « indexées par le temps » (où l'horizon temporel est discrétisé en unités de temps) servent de fondement et de référence comparative majeure dans l'état de l'art pour évaluer d'autres paradigmes [4]. Ces modèles linéaires en nombres entiers sont également utilisés comme briques de base dans des méthodes hybrides de décomposition, que ce soit pour formuler le problème maître de Benders [17, 18] ou pour concevoir des approches de génération de colonnes [5, 6].

Dans ce chapitre, nous posons les bases théoriques de la PLNE, décrivons les mécanismes généraux de résolution sous-jacents aux solveurs commerciaux, puis détaillons la modélisation mathématique du problème du MS-RCPSP-AF d'après les travaux de Polo-Mejía et al. [8, 19, 20].

1.1 Structure mathématique d'un modèle PLNE

Un programme linéaire en nombres entiers se structure formellement autour de trois composants principaux :

Les variables de décision : Les variables de décision sont les inconnues du modèle d'optimisation. Elles traduisent les décisions à prendre et déterminent la solution du problème. En programmation linéaire en nombres entiers, ces variables ne peuvent prendre que des valeurs entières, et parfois uniquement les valeurs binaires 0 ou 1 pour représenter des choix de type « oui/non ».

La fonction objectif : Une fonction mathématique linéaire que le solveur doit optimiser (généralement la minimisation du coût total ou du temps de fin de projet, le Makespan). Elle s'exprime sous la forme d'une somme de variables pondérées par des coûts.

Les contraintes : Un ensemble d'équations et d'inégalités strictes qui limitent l'espace des solutions réalisables. Elles traduisent les capacités de production, les relations de précédence ou les limites de ressources. Elles doivent être linéaires : les multiplications de variables entre elles ou les conditions logiques directes y sont interdites.

1.2 Mécanismes de résolution :

Contrairement à la programmation linéaire continue, qui se résout très rapidement en temps polynomial à l'aide de l'algorithme du Simplexe en exploitant la géométrie des volumes convexes, la condition d'intégralité des variables transforme le problème en un défi combinatoire hautement complexe (NP-difficile). L'espace des solutions réalisables se restreint alors à un nuage de points discrets et isolés.

Pour explorer cet espace de manière exacte, les solveurs industriels modernes (tels que Gurobi ou IBM ILOG CPLEX) combinent la programmation linéaire avec des algorithmes d'exploration arborescente, principalement la méthode de Séparation et Évaluation (Branch-and-Bound) et le Branch-and-Cut.

La relaxation continue : À chaque nœud de l'arbre de recherche, le solveur résout le problème en omettant temporairement la contrainte d'intégralité des variables. Cette phase fournit une borne inférieure idéale du problème de minimisation.

Branching : Si une variable devant être entière prend une valeur fractionnaire (par exemple 2.4) dans la relaxation continue, le solveur sépare l'espace de recherche en créant deux sous-nœuds enfants enrichis de nouvelles contraintes restrictives : l'une imposant que la variable soit inférieure ou égale à 2, et l'autre supérieure ou égale à 3.

L'évaluation et l'élagage (Bounding and Pruning) : L'arbre est exploré de manière itérative. Si la borne continue calculée à un nœud donné est mathématiquement moins bonne qu'une solution entière valide déjà mémorisée par le solveur (la borne supérieure), le solveur coupe immédiatement ce nœud et l'ensemble de sa descendance, s'évitant ainsi une exploration combinatoire exhaustive et inutile.

1.3 Application au problème du MS-RCPSP-AF

Afin de poser les bases mathématiques de notre problème et d'en analyser la complexité théorique, nous allons formaliser le problème du MS-RCPSP-AF sous la forme d'un modèle en Programmation Linéaire en Nombres Entiers, ouvrant la voie aux choix de modélisations effectives par Programmation par Contraintes.

1.3.1 Ensembles

- $\mathcal{A}$: Ensemble des activités à planifier.
- $\mathcal{L}$: Ensemble des différentes compétences.
- $\mathcal{W}$: Ensemble des ressources multi-compétences (travailleurs).
- $\mathcal{CR}$: Ensemble des ressources cumulatives.
- $\mathcal{T}$: Horizon temporel, $\mathcal{T} = \{1, \dots, T\}$, indexé par t .
- E : Ensemble des contraintes de précédence.

1.3.2 Paramètres

- p_i : Temps de traitement de l'activité i .
- B_k : Quantité de ressource k .
- $b_{i,k}$: Quantité de ressource k requise pour l'activité i .
- $a_{i,l}$: Nombre de travailleurs maîtrisant la compétence l requis pour l'activité i .
- $m_{w,l}$: Égal à 1 si le travailleur w maîtrise la compétence l , 0 sinon.
- q_i : Nombre minimum de travailleurs requis pour l'activité i .
- r_i, d_i : Date de disponibilité et date d'échéance autorisées pour l'activité i .

1.3.3 Variables de décision

- $x_{i,t} \in \{0, 1\}$: Égal à 1 si l'activité i est exécutée à l'instant t , 0 sinon.
- $z_{w,i,l,t} \in \{0, 1\}$: Égal à 1 si le travailleur w est affecté à l'activité i pour exercer la compétence l à l'instant t , 0 sinon.

1.3.4 Fonction objectif

L'objectif est de minimiser la durée totale d'exécution du projet (*Makespan*) :

$$\min C_{\max} \quad (1)$$

1.3.5 Contraintes

Unicité stricte d'affectation : Un travailleur peut être affecté au maximum à une seule activité ET une seule compétence simultanément à l'instant t .

$$\sum_{i \in \mathcal{A}} \sum_{l \in \mathcal{L}} z_{w,i,l,t} \leq 1 \quad \forall w \in \mathcal{W}, \forall t \in \mathcal{T} \quad (2)$$

Validation de la maîtrise des compétences : Le modèle ne peut affecter un travailleur à une compétence que si celui-ci possède effectivement cette capacité spécifique.

$$z_{w,i,l,t} \leq m_{w,l} \quad \forall w \in \mathcal{W}, \forall i \in \mathcal{A}, \forall l \in \mathcal{L}, \forall t \in \mathcal{T} \quad (3)$$

Couverture des exigences techniques en compétences : Si une activité est active ($x_{i,t} = 1$), la somme des travailleurs exerçant explicitement la compétence l doit satisfaire le besoin requis $a_{i,l}$.

$$\sum_{w \in \mathcal{W}} z_{w,i,l,t} \geq a_{i,l} \cdot x_{i,t} \quad \forall l \in \mathcal{L}, \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (4)$$

Respect de l'effectif minimum requis pour une activité : Pour comptabiliser l'effectif total affecté à l'activité i à l'instant t , nous sommions toutes les affectations par compétence pour cette activité. Ce total doit satisfaire l'exigence physique q_i .

$$\sum_{w \in \mathcal{W}} \sum_{l \in \mathcal{L}} z_{w,i,l,t} \geq q_i \cdot x_{i,t} \quad \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (5)$$

Respect des durées de traitement : Chaque activité doit être exécutée pendant exactement p_i périodes à l'intérieur de sa fenêtre de temps admissible.

$$\sum_{t=r_i}^{d_i} x_{i,t} = p_i \quad \forall i \in \mathcal{A} \quad (6)$$

Contraintes de précédence : Une activité successeur j ne peut pas démarrer tant que son activité prédécesseur i n'est pas complètement achevée.

$$p_i \cdot (1 - x_{j,t}) \geq \sum_{t'=t}^T x_{i,t'} \quad \forall (i, j) \in E, \forall t \in \mathcal{T} \quad (7)$$

Capacité des ressources matérielles cumulatives : Puisque les tâches ne peuvent pas être interrompues, l'utilisation des ressources matérielles se simplifie en une contrainte de capacité standard.

$$\sum_{i \in \mathcal{A}} b_{i,k} \cdot x_{i,t} \leq B_k \quad \forall k \in \mathcal{CR}, \forall t \in \mathcal{T} \quad (8)$$

Définition de la borne inférieure du Makespan

$$C_{\max} \geq t \cdot x_{i,t} \quad \forall i \in \mathcal{A}, \forall t \in \mathcal{T} \quad (9)$$

Cette contrainte est indispensable pour lier la variable d'objectif $C_{\max}$ au déroulement temporel du projet. Le modèle étant indexé par le temps, si une activité i est en cours d'exécution à l'instant

t , la variable binaire associée s'active ($x_{i,t} = 1$). L'inéquation force alors le $C_{\max}$ à être supérieur ou égal à cet instant t . Par conséquent, lors de la phase de minimisation de la fonction objectif, la variable $C_{\max}$ est poussée vers le bas jusqu'à se caler exactement sur la valeur du tout dernier instant t où une activité est encore active dans le projet, capturant ainsi la date de fin globale.

1.4 Analyse des limites et verrous calculatoires de la formulation PLNE

Bien que le modèle PLNE ci-dessus fournisse une formulation mathématique exacte et rigoureuse du problème du MS-RCPSP-AF, il souffre de verrous calculatoires critiques qui empêchent sa résolution sur des instances de taille moyenne ou grande.

1.4.1 Explosion dimensionnelle avec l'horizon temporel

La principale limite de cette approche réside dans l'utilisation de variables indexées par le temps ($x_{i,t}$ et $z_{w,i,l,t}$). Pour un projet ayant un horizon temporel T (qui correspond à la borne supérieure de la durée du projet), le nombre de variables de décision binaires est proportionnel à T . Le nombre de variables d'affectation $z_{w,i,l,t}$ s'élève à $|\mathcal{W}| \times |\mathcal{A}| \times |\mathcal{L}| \times T$. Si T est grand (ce qui est fréquent dans des projets industriels avec des durées de tâches hétérogènes), le solveur doit gérer des millions de variables binaires, ce qui sature rapidement la mémoire et rend la phase de séparation (*Branching*) impraticable.

1.4.2 Faiblesse de la relaxation continue

Les formulations indexées par le temps pour l'ordonnancement de projets sont connues pour avoir une relaxation continue très faible. Lorsque les contraintes d'intégralité ($x_{i,t}, z_{w,i,l,t} \in \{0, 1\}$) sont relâchées dans la relaxation continue ($0 \leq x_{i,t}, z_{w,i,l,t} \leq 1$), le solveur linéaire trouve des solutions fractionnaires triviales où les tâches sont exécutées de manière fragmentée (par exemple, 10 % d'une tâche à l'instant 1, 10 % à l'instant 10, etc.). La borne inférieure fournie par cette relaxation est donc extrêmement éloignée de la réalité entière. En conséquence, le solveur est contraint d'explorer un nombre gigantesque de nœuds dans l'arbre de recherche de *Branch-and-Bound* pour prouver l'optimalité, conduisant à des temps de calcul prohibitifs.

1.4.3 Linéarisation des relations logiques

Pour conserver le formalisme linéaire, il est nécessaire de linéariser toutes les contraintes logiques. C'est le cas des contraintes de précédence (7) :

$$p_i \cdot (1 - x_{j,t}) \geq \sum_{t'=t}^T x_{i,t'} \quad \forall (i, j) \in E, \forall t \in \mathcal{T} \quad (10)$$

Si le successeur j est en cours d'exécution à l'instant t ($x_{j,t} = 1$), le membre de gauche s'annule, ce qui force la somme du membre de droite à être égale à 0. Ainsi, le prédécesseur i ne peut plus être en cours d'exécution à partir de cet instant t , garantissant que i est entièrement terminé avant le début de j . Si j n'est pas actif à l'instant t ($x_{j,t} = 0$), la contrainte devient triviale ($p_i \geq \sum x_{i,t'}$) car la somme des périodes actives de i ne peut de toute façon pas dépasser sa durée p_i . Bien que cette linéarisation soit élégante, elle doit être écrite pour chaque période t et chaque arc du graphe de précédence E , augmentant considérablement le nombre de contraintes du modèle.

2 Programmation par Contraintes

La Programmation par Contraintes (PPC, ou CP pour *Constraint Programming* en anglais) est un paradigme déclaratif d'optimisation exacte qui a émergé dans les années 1970 et 1980 à la croisée de l'intelligence artificielle et de la recherche opérationnelle. Particulièrement adapté à la résolution de problèmes combinatoires complexes comme la planification et l'ordonnancement, ce paradigme se distingue de la programmation linéaire par sa capacité à modéliser des contraintes non linéaires, logiques et globales.

En PPC, l'utilisateur se concentre uniquement sur la modélisation du problème, tandis que la recherche de solutions et la preuve d'optimalité sont déléguées à un solveur générique. Ce dernier repose sur l'intégration étroite de la propagation de contraintes et de l'exploration arborescente. Une des grandes forces de la PPC réside dans le concept de contraintes globales, qui encapsulent des structures combinatoires récurrentes (comme l'ordonnancement de tâches ou l'affectation). À chaque contrainte globale est associé un propagateur (algorithme de filtrage) très performant. Pour l'ordonnancement sous contraintes cumulatives, ces propagateurs mettent en œuvre des algorithmes avancés de l'état de l'art tels que le *timetabling* [21], le *cumulative edge-finding* [22] ou le raisonnement énergétique [23].

L'application de la PPC au problème de l'ordonnancement de projet multi-compétences (MSPSP) s'est avérée particulièrement fructueuse. Des approches basées sur le CP/SAT, le *lazy clause generation* et l'apprentissage de no-goods ont été développées par Young et al. [7]. Pour le MS-RCPSP avec gestion fine de la préemption et des contraintes de ressources complexes, Polo-Mejía et al. [8, 19] ont proposé des modèles en programmation par contraintes exploitant l'efficacité des variables d'intervalles natives de solveurs industriels comme *IBM ILOG CP Optimizer*. De plus, des travaux comme ceux de Carlier et Néron [24] ou de Baptiste et Bonifas [25] ont démontré l'intérêt d'injecter des enveloppes cumulatives redondantes pour accélérer la convergence temporelle.

Dans ce chapitre, nous présentons le formalisme général des CSP, décrivons les primitives d'ordonnancement spécialisées de CP Optimizer, formulons le modèle CP monolithique pour le MS-RCPSP-AF, et discutons enfin de ses limites combinatoires.

2.1 Formalisme des Problèmes de Satisfaction de Contraintes (CSP)

Contrairement à la Programmation Linéaire qui requiert une formulation sous forme de relations arithmétiques linéaires, la PPC repose sur le formalisme des Problèmes de Satisfaction de Contraintes (CSP, pour *Constraint Satisfaction Problems*). Un CSP est défini formellement par un triplet (X, D, C) où :

- $X = \{x_1, x_2, \dots, x_n\}$ est l'ensemble des variables de décision du problème.
- $D = \{D(x_1), D(x_2), \dots, D(x_n)\}$ représente l'ensemble des domaines associés à chaque variable, c'est-à-dire l'ensemble des valeurs admissibles pour chaque variable (généralement des ensembles discrets d'entiers).
- $C = \{c_1, c_2, \dots, c_m\}$ est l'ensemble des contraintes. Une contrainte c_j porte sur un sous-ensemble de variables et restreint les combinaisons de valeurs que ces variables peuvent prendre simultanément.

L'objectif consiste à affecter à chaque variable x_i une valeur de son domaine $D(x_i)$ de manière à satisfaire l'intégralité des contraintes de C . Dans le cas d'un problème d'optimisation sous contraintes (COP), on cherche de plus à minimiser ou maximiser une fonction objectif $f(X)$.

2.2 Mécanismes de résolution : Propagation et Recherche

Pour explorer l'espace des solutions, les solveurs modernes de PPC (comme *IBM ILOG CP Optimizer*) n'effectuent pas une recherche exhaustive aveugle, mais couplent dynamiquement deux mécanismes :

1. **La propagation de contraintes (filtrage) :** À chaque contrainte $c_j \in C$ est associé un algorithme de filtrage (ou propagateur). Lorsqu'une décision restreint le domaine d'une variable, le propagateur évalue la cohérence locale (telle que la cohérence d'arc ou *Arc Consistency*) et élimine des domaines des autres variables connectées toutes les valeurs incompatibles avec la contrainte. Ce mécanisme d'inférence réduit activement la taille des domaines en cours de route.
2. **La recherche arborescente (exploration) :** Lorsque la propagation seule ne suffit pas à déterminer une solution unique, le solveur effectue un branchement (*branching*) : il choisit une variable dont le domaine contient encore plusieurs valeurs, fixe cette variable à l'une d'entre elles, puis relance la propagation. Si un conflit est détecté (un domaine devient vide), le solveur effectue un retour arrière (*backtracking*) vers le dernier point de branchement et essaie une autre valeur. Ce couplage itératif entre propagation et recherche garantit l'exhaustivité de l'exploration et permet de prouver l'optimalité.

2.3 Primitives d'ordonnancement de CP Optimizer

Pour résoudre efficacement les problèmes d'ordonnancement de projets, les solveurs modernes comme *IBM ILOG CP Optimizer* introduisent des primitives de modélisation spécifiques de haut niveau, qui évitent d'avoir à discrétiser explicitement le temps comme en PLNE.

2.3.1 Variables d'intervalles (Interval Variables)

Une variable d'intervalle, notée a , représente une tâche ou une portion de tâche caractérisée par une date de début $a.start$, une date de fin $a.end$, et une durée $a.size$ (ou $a.length$). Ces valeurs sont des variables entières vérifiant la relation $a.end = a.start + a.size$. De plus, une variable d'intervalle peut être définie comme **optionnelle** : le solveur décide alors si l'intervalle est présent ou non dans la solution finale ($a.present \in \{0, 1\}$). Si un intervalle est absent, ses attributs de temps (début, fin, taille) sont ignorés par les contraintes qui le lient aux autres intervalles.

2.3.2 Contraintes temporelles

Les relations de précédence et de synchronisation entre activités s'expriment à l'aide de contraintes spécialisées :

- **endBeforeStart(a, b, z) :** Impose que le début de l'intervalle b soit postérieur ou égal à la fin de l'intervalle a , avec éventuellement un délai de transition z :

$$b.start \geq a.end + z \quad (11)$$

Cette contrainte n'est active que si les deux intervalles a et b sont présents dans la solution.

- **startAtEnd(a, b) :** Impose que l'intervalle a commence exactement au moment où l'intervalle b se termine :

$$a.start = b.end \quad (12)$$

Cette contrainte est indispensable pour modéliser des jonctions temporelles strictes (sans interruption).

- **span($a, \{b_1, \dots, b_k\}$) :** Force l'intervalle a à couvrir exactement l'ensemble des intervalles présents dans le groupe $\{b_1, \dots, b_k\}$. Ainsi, a débute au début du premier intervalle actif du groupe et se termine à la fin du dernier intervalle actif du groupe :

$$a.start = \min_{1 \leq i \leq k} \{b_i.start \mid b_i.present = 1\} \quad (13)$$

$$a.\text{end} = \max_{1 \leq i \leq k} \{b_i.\text{end} \mid b_i.\text{present} = 1\} \quad (14)$$

- **alternative(a, {b₁, ..., b_k}, c)** : Associe un intervalle global a à un ensemble de sous-intervalles optionnels $\{b_1, \dots, b_k\}$. Si a est présent, alors exactement c sous-intervalles parmi $\{b_1, \dots, b_k\}$ doivent être présents, et ces c sous-intervalles sélectionnés doivent débuter et finir exactement en même temps que a . Dans sa version standard ($c = 1$), cette contrainte permet de modéliser le choix d'une ressource unique parmi plusieurs alternatives.

2.3.3 Ressources disjonctives et cumulatives

- **Non-chevauchement (noOverlap)** : Pour modéliser une ressource disjonctive (comme un travailleur individuel qui ne peut exécuter qu'une seule tâche à la fois), on regroupe toutes les variables d'intervalles affectées à cette ressource dans une contrainte **noOverlap**. Cette contrainte garantit que deux intervalles quelconques du groupe ne peuvent pas se chevaucher dans le temps si les deux sont présents.

2.4 Modélisation CP monolithique pour le MS-RCPSP-AF

Afin de capturer la flexibilité d'affectation (rotation des équipes) sans préemption de la tâche globale, nous développons un modèle par Programmation par Contraintes fondé sur la discrétisation interne des activités en parties unitaires.

2.4.1 Ensembles et paramètres

Le modèle réutilise les ensembles de base introduits au Chapitre 1 ($\mathcal{A}, \mathcal{L}, \mathcal{W}, \mathcal{CR}, E$) et y adjoint l'ensemble suivant :

- $V_i = \{1, \dots, p_i\}$: Ensemble des parties de durée unitaire de l'activité $i \in \mathcal{A}$, indexé par v .

Nous définissons également le paramètre supplémentaire suivant :

- N_l : Nombre total de travailleurs maîtrisant la compétence l dans l'ensemble $\mathcal{W}$.

2.4.2 Variables de décision

Le modèle s'appuie sur la définition de variables d'intervalles à plusieurs niveaux :

- $C_{\max} \geq 0$: Variable entière représentant la date de fin globale du projet.
- act_i : Variable d'intervalle représentant l'exécution globale de l'activité i . Sa taille est fixée à p_i .
- $par_{i,v}$: Variable d'intervalle de durée unitaire représentant la v -ème partie de l'activité i (pour $v \in V_i$).
- $awo_{w,i,l,v}$: Variable d'intervalle optionnelle de durée unitaire représentant l'affectation du travailleur w exerçant la compétence l pour la v -ème partie de l'activité i .

2.4.3 Formulation mathématique

Le modèle CP monolithique s'écrit de la manière suivante :

$$\min \quad C_{\max} \quad (15)$$

s.t.

$$C_{\max} \geq \text{act}_i.\text{end} \quad \forall i \in \mathcal{A} \quad (16)$$

$$\text{span}(\text{act}_i, \{\text{par}_{i,v} \mid v \in V_i\}) \quad \forall i \in \mathcal{A} \quad (17)$$

$$\text{startAtEnd}(\text{par}_{i,v+1}, \text{par}_{i,v}) \quad \forall i \in \mathcal{A}, \forall v \in \{1, \dots, p_i - 1\} \quad (18)$$

$$\text{endBeforeStart}(\text{act}_i, \text{act}_m) \quad \forall (i, m) \in E \quad (19)$$

$$\sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, b_{i,k}) \leq B_k \quad \forall k \in \mathcal{CR} \quad (20)$$

$$\text{noOverlap}(\{\text{awo}_{w,i,l,v} \mid \forall i \in \mathcal{A}, \forall l \in \mathcal{L}, \forall v \in V_i\}) \quad \forall w \in \mathcal{W} \quad (21)$$

$$\text{alternative}(\text{par}_{i,v}, \{\text{awo}_{w,i,l,v} \mid \forall w \in \mathcal{W} \text{ s.t. } m_{w,l} = 1\}, a_{i,l}) \quad \forall i \in \mathcal{A}, \forall v \in V_i, \forall l \in \mathcal{L} \quad (22)$$

$$\text{alternative}(\text{par}_{i,v}, \{\text{awo}_{w,i,l,v} \mid \forall w \in \mathcal{W}, \forall l \in \mathcal{L}\}, q_i) \quad \forall i \in \mathcal{A}, \forall v \in V_i \quad (23)$$

2.4.4 Explication des contraintes

- L'équation (16) définit le makespan à minimiser comme étant supérieur ou égal à la fin de chaque activité globale act_i .
- La contrainte de couverture (17) lie l'activité globale act_i à l'ensemble de ses sous-intervalles unitaires $\text{par}_{i,v}$.
- L'équation (18) assure la contiguïté temporelle (non-préemption) en forçant la fin de la partie v à coïncider exactement avec le début de la partie $v + 1$.
- La contrainte de précédence (19) assure les relations d'ordre logique du projet entre les activités globales.
- L'équation (20) restreint l'usage simultané des ressources matérielles cumulatives à leur capacité B_k .
- L'équation (21) garantit la disjonction de chaque travailleur : un opérateur w ne peut pas travailler sur plusieurs tâches ou plusieurs compétences en même temps.
- La contrainte alternative (22) est le cœur de la flexibilité : pour chaque partie unitaire v de l'activité i et pour chaque compétence l , elle force la sélection d'exactly $a_{i,l}$ intervalles optionnels d'affectation parmi les travailleurs qualifiés. Ces travailleurs sélectionnés travailleront sur le même intervalle temporel unitaire.
- L'équation (23) fait de même pour satisfaire la contrainte d'effectif physique minimum global q_i sur chaque partie unitaire.

2.4.5 Contraintes redondantes de capacité globale

Afin d'accélérer la propagation et d'aider le solveur à détecter plus rapidement les surcharges de capacité globale de main-d'œuvre, nous introduisons des enveloppes cumulatives globales redondantes :

$$wU = \sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, q_i) \leq |\mathcal{W}| \quad (24)$$

$$sU_l = \sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, a_{i,l}) \leq N_l \quad \forall l \in \mathcal{L} \quad (25)$$

Ces enveloppes ne modifient pas l'espace des solutions, mais elles renforcent les mécanismes de filtrage en évaluant l'agrégation de la demande de main-d'œuvre au niveau des activités globales act_i plutôt que sur chaque partie unitaire.

2.5 Contraintes de bris de symétrie

Le modèle CP monolithique présente d'importantes symétries de permutation. Si plusieurs travailleurs possèdent les mêmes compétences, les solutions qui consistent à permuter ces travailleurs sur des tâches identiques sont équivalentes en termes de coût (Makespan), mais elles obligent le solveur à explorer des branches redondantes dans l'arbre de recherche.

Afin de briser la symétrie, nous introduisons une contrainte forçant la conservation de l'affectation des travailleurs entre les unités de temps, sauf lorsqu'un événement majeur (comme le début d'une nouvelle tâche) se produit. En effet, les travailleurs n'ont de raisons de changer d'affectation ou de tourner que lorsqu'une nouvelle activité démarre et libère ou requiert de nouvelles compétences. Entre deux démarrages d'activités, l'affectation doit rester constante, ce qui élimine les rotations inutiles et équivalentes.

La contrainte s'exprime ainsi :

$$\text{IfThen}(PaS - \text{pulse}(par_{i,v}, 1) \leq -1, awo_{w,i,l,v} = awo_{w,i,l,v-1}) \quad \forall l \in \mathcal{L}, \forall w \in \mathcal{W}, \forall i \in \mathcal{A}, \forall v \in V_i \setminus \{0\} \quad (26)$$

Avec :

$$PaS = \sum_{i \in \mathcal{A}} \text{pulse}(par_{i,0}, 1) \quad (27)$$

Qui vaut zéro partout sauf lorsqu'au moins une tâche débute (c'est-à-dire lors de l'exécution de la première partie unitaire $par_{i,0}$ de n'importe quelle activité i).

Ainsi, si une partie unitaire $par_{i,v}$ ($v \geq 1$) est active alors qu'aucune tâche ne débute à cet instant ($PaS = 0$, donc $PaS - \text{pulse}(par_{i,v}, 1) = -1$), l'affectation des travailleurs pour la compétence l sur cette partie v est forcée d'être identique à celle de la partie précédente $v-1$ ($awo_{w,i,l,v} = awo_{w,i,l,v-1}$).

Bien que cette contrainte réduise théoriquement l'espace de recherche en éliminant les permutations équivalentes et les rotations superflues, l'évaluation expérimentale (présentée au Chapitre 4) montre qu'elle dégrade en réalité les performances globales du solveur CP Optimizer. Cela s'explique par deux facteurs :

- **L'overhead de propagation :** L'ajout de ces contraintes logiques conditionnelles alourdit la charge de travail du moteur de propagation à chaque nœud de l'arbre de recherche.
- **Perturbation des heuristiques de recherche :** Les solveurs modernes de PPC intègrent des heuristiques de recherche dynamique extrêmement avancées (comme le *Failure-Directed Search* dans CP Optimizer). L'imposition rigide de contraintes de bris de symétrie peut entrer en conflit avec ces heuristiques internes en masquant des chemins d'apprentissage de conflits.

2.6 Analyse des limites et verrous combinatoires du modèle CP monolithique

Bien que cette formulation CP monolithique décrive de manière très naturelle le problème du MS-RCPSP-AF grâce aux variables d'intervalles unitaires et aux contraintes alternatives, elle souffre de sévères verrous combinatoires qui empêchent son passage à l'échelle sur des instances industrielles.

2.6.1 Explosion du nombre de variables

La flexibilité d'affectation impose de valider la faisabilité à chaque unité de temps. La discrétisation en intervalles unitaires de durée 1 multiplie le nombre de variables par la durée des activités. Pour chaque activité i , on crée p_i intervalles unitaires. Pour chaque partie unitaire $par_{i,v}$, on doit

définir un intervalle optionnel $awo_{w,i,l,v}$ pour chaque travailleur w maîtrisant la compétence l . La quantité de variables d'intervalles optionnelles s'élève à :

$$\sum_{i \in \mathcal{A}} p_i \sum_{l \in \mathcal{L}} \sum_{w \in \mathcal{W}} m_{w,l} \quad (28)$$

Sur des instances comportant quelques dizaines de tâches de durées moyennes importantes et des dizaines de travailleurs qualifiés, le nombre de variables optionnelles dépasse rapidement les dizaines de milliers. Les domaines de recherche deviennent gigantesques, saturant le moteur de propagation et la mémoire du solveur.

2.6.2 Symétries massives

Le modèle présente une forte symétrie de permutation des travailleurs. Si deux travailleurs w_1 et w_2 possèdent le même ensemble de compétences, les affectations permutant w_1 et w_2 sur une même tâche sont mathématiquement équivalentes mais explorées comme distinctes par l'arbre de recherche du solveur. Bien que CP Optimizer dispose de heuristiques de recherche pour limiter l'impact des symétries, celles-ci ralentissent fortement la preuve d'optimalité.

Ces limitations majeures motivent le développement d'une méthode de décomposition, la Décomposition de Benders Logique (LBBD), que nous détaillerons au Chapitre 3, afin de découpler la dimension temporelle de la dimension d'affectation spatiale des travailleurs.

3 Décomposition de Benders basée sur la Logique

Pour surmonter l’explosion combinatoire et les limites du modèle CP monolithique sur des instances industrielles de grande taille, ce chapitre présente une méthode de décomposition hybride exacte : la Décomposition de Benders basée sur la Logique (LBBD, pour *Logic-Based Benders Decomposition*). Cette approche découple intelligemment les décisions temporelles d’ordonnancement (quand exécuter les tâches) des décisions spatiales d’affectation de main-d’œuvre (qui fait quoi à chaque instant).

La décomposition de Benders classique, introduite par Jacques Benders [26], est une technique de programmation mathématique qui partitionne les variables d’un problème en deux sous-ensembles (maître et sous-problème) résolus de manière itérative. Le sous-problème doit obligatoirement être un programme linéaire continu afin de pouvoir exploiter les variables duales et générer des coupes d’optimalité ou de réalisabilité. La Décomposition de Benders basée sur la Logique (LBBD), introduite formellement par John Hooker [9, 10], généralise ce principe aux cas où le sous-problème est un problème combinatoire quelconque (par exemple, résolu en Programmation par Contraintes ou par des algorithmes de graphes). Les coupes ne sont plus issues de la dualité linéaire classique, mais sont des conditions logiques formulées à partir de la preuve d’infaisabilité ou d’optimalité du sous-problème.

Le framework LBBD s’est avéré particulièrement performant pour les problèmes intégrés d’ordonnancement et de planification, comme l’atteste l’analyse computationnelle approfondie menée par Cire et al. [11]. Dans la littérature du MSPSP, les premiers schémas de Benders proposaient un partitionnement classique (le problème maître MILP résout l’affectation globale et le sous-problème CP gère l’ordonnancement temporel), comme dans les travaux de Li et Womer [17] et de Li [18], ou encore pour la planification stochastique des équipes par Nasirian et al. [27]. Pour remédier au manque d’informations d’ordonnancement dans le problème maître, Yi-fan et al. [12] ont proposé d’inverser ce schéma (Master CP pour l’ordonnancement et Sous-problème d’affectation). Récemment, Juvin et al. [13] ont étendu ce schéma inverse pour intégrer la flexibilité d’affectation dynamique (MS-RCPSP-AF).

Dans notre cas, la décomposition s’articule ainsi :

- **Le Problème Maître (CP) :** Il planifie les activités globales sur un horizon temporel continu en respectant les précédences et des relaxations globales de capacité de main-d’œuvre. L’inclusion de ces relaxations dans le problème maître est essentielle pour guider la recherche et réduire le nombre d’itérations maître/esclave [11].
- **Le Sous-Problème d’Affectation (Flot Max) :** Pour un ordonnancement fixé par le maître, il vérifie de manière indépendante pour chaque période t s’il existe une affectation valide des travailleurs pour couvrir les besoins en compétences des activités actives. En cas d’infaisabilité, l’extraction de coupes s’apparente à l’identification d’ensembles critiques ou interdits (*forbidden sets*) [28, 29].

3.1 Principes de la Décomposition de Benders basée sur la Logique (LBBD)

La Décomposition de Benders basée sur la Logique repose sur le fait de projeter le problème sur le sous-ensemble des variables d’ordonnancement (le problème maître) et de n’évaluer la faisabilité de l’affectation (le sous-problème) que sur les solutions candidates générées.

3.2 Problème Maître d’Ordonnancement

Le Problème Maître est une version relaxée du modèle CP monolithique présenté au Chapitre 2. Il se concentre exclusivement sur l’aspect temporel de l’ordonnancement global des tâches. Il ignore l’affectation nominative des opérateurs, mais intègre des contraintes de capacité cumulative globale (enveloppes de main-d’œuvre) pour éviter de générer des plannings manifestement irréalisables.

3.2.1 Formulation mathématique

Soit Ω l'ensemble des coupes de Benders générées et injectées dynamiquement au cours des itérations. Le Problème Maître s'exprime sous la forme du modèle CP suivant :

$$\min \quad C_{\max} \quad (29)$$

s. t.

$$C_{\max} \geq \text{act}_i.\text{end} \quad \forall i \in \mathcal{A} \quad (30)$$

$$\text{endBeforeStart}(\text{act}_i, \text{act}_m) \quad \forall (i, m) \in E \quad (31)$$

$$\sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, b_{i,k}) \leq B_k \quad \forall k \in \mathcal{CR} \quad (32)$$

$$\sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, q_i) \leq |\mathcal{W}| \quad (33)$$

$$\sum_{i \in \mathcal{A}} \text{pulse}(\text{act}_i, a_{i,l}) \leq N_l \quad \forall l \in \mathcal{L} \quad (34)$$

$$\text{Contraintes de coupes de Benders (injections de } \Omega) \quad (35)$$

Les équations (30) à (34) correspondent aux contraintes d'ordonnancement classiques et aux relaxations de capacité cumulatives (la somme de l'effectif minimum physique requis ne doit pas dépasser le nombre total de travailleurs $|\mathcal{W}|$, et la demande cumulée pour chaque compétence l ne doit pas dépasser le nombre d'opérateurs qualifiés N_l). La contrainte (35) représente les coupes cumulatives dynamiques générées par les sous-problèmes d'affectation pour interdire les conflits de compétences détectés.

3.3 Sous-problème d'Affectation

À chaque itération, le solveur résout le Problème Maître et obtient un ordonnancement partiel où les dates de début et de fin de chaque activité globale act_i sont fixées. À partir de cet ordonnancement, nous pouvons déterminer pour chaque période $t \in \mathcal{T}$ l'ensemble des activités actives :

$$\mathcal{A}(t) = \{i \in \mathcal{A} \mid \text{act}_i.\text{start} \leq t < \text{act}_i.\text{end}\} \quad (36)$$

Pour chaque période t , la demande totale pour la compétence l , notée $D_l(t)$, est la somme des besoins des tâches actives à cet instant :

$$D_l(t) = \sum_{i \in \mathcal{A}(t)} a_{i,l} \quad \forall l \in \mathcal{L} \quad (37)$$

La validation de la faisabilité spatiale consiste à vérifier s'il existe une affectation binaire des travailleurs $w \in \mathcal{W}$ aux compétences $l \in \mathcal{L}$ telle que la demande $D_l(t)$ soit satisfaite pour toute compétence et que chaque travailleur ne soit affecté qu'au plus à une seule compétence.

3.3.1 Formulation sous forme de Flot Maximum (Max-Flow)

Puisque les périodes sont indépendantes une fois le temps fixé, le problème d'affectation pour une période t donnée peut être modélisé très efficacement comme un problème de Flot Maximum dans un réseau de transport.

Nous construisons un graphe orienté $G(V, A)$ composé de :

- Une source s et un puits p .
- Un nœud pour chaque compétence $l \in \mathcal{L}$.

- Un nœud pour chaque travailleur $w \in \mathcal{W}$.

Les arcs et leurs capacités associées sont définis comme suit :

- Un arc orienté de la source s vers chaque nœud de compétence l avec une capacité égale à la demande $D_l(t)$.
- Un arc orienté de chaque nœud de compétence l vers chaque nœud de travailleur w si et seulement si le travailleur maîtrise cette compétence ($m_{w,l} = 1$). La capacité de cet arc est infinie.
- Un arc orienté de chaque nœud de travailleur w vers le puits p avec une capacité unitaire (1), pour garantir qu'un travailleur ne peut être affecté qu'à une seule tâche/compétence.

Le problème de flot maximum se formule ainsi :

$$\max \sum_{l \in \mathcal{L}} f_{s,l} \quad (38)$$

$$\text{s.t. } f_{s,l} \leq D_l(t) \quad \forall l \in \mathcal{L} \quad (39)$$

$$f_{s,l} - \sum_{w \in \mathcal{W}} f_{l,w} \cdot m_{w,l} = 0 \quad \forall l \in \mathcal{L} \quad (40)$$

$$\sum_{l \in \mathcal{L}} f_{l,w} - f_{w,p} = 0 \quad \forall w \in \mathcal{W} \quad (41)$$

$$f_{w,p} \leq 1 \quad \forall w \in \mathcal{W} \quad (42)$$

$$f \geq 0 \quad (43)$$

Si la valeur du flot maximum optimal est strictement inférieure à la demande totale $\sum_{l \in \mathcal{L}} D_l(t)$, alors l'affectation est irréalisable à l'instant t . Le solveur doit alors générer une coupe de Benders pour exclure cet ordonnancement temporel.

3.4 Génération des coupes de Benders

Lorsqu'une infaisabilité est détectée à une période t , nous devons identifier la cause de cette infaisabilité sous la forme d'un conflit de compétences, puis exprimer ce conflit sous forme de contrainte globale (coupe) à réinjecter dans le Problème Maître CP.

3.4.1 Formulation duale et Coupe Minimale (Min-Cut)

Pour identifier le sous-ensemble minimal de compétences à l'origine du conflit, nous exploitons la dualité linéaire. Le problème dual associé au problème d'affectation (ou le théorème du flot maximum/coupe minimale) permet d'identifier une coupe minimale dans le réseau.

Le modèle dual se formule ainsi :

$$\min \sum_{w \in \mathcal{W}} d_{w,p} + \sum_{l \in \mathcal{L}} D_l(t) \cdot d_{s,l} \quad (44)$$

$$\text{s.t. } d_{s,l} + p_l \geq 1 \quad \forall l \in \mathcal{L} \quad (45)$$

$$d_{w,p} + p_w \geq 0 \quad \forall w \in \mathcal{W} \quad (46)$$

$$p_w - m_{w,l} \cdot p_l \geq 0 \quad \forall l \in \mathcal{L}, \forall w \in \mathcal{W} \quad (47)$$

Soit $S \subseteq \mathcal{L}$ le sous-ensemble de compétences situées du côté source de la coupe minimale dans la solution duale optimale (c'est-à-dire les compétences pour lesquelles $d_{s,l} = 0$). Soit W_S le sous-ensemble de travailleurs connectés à ces compétences du côté source de la coupe ($d_{w,p} = 1$). D'après

le théorème de la coupe minimale, si l'affectation est infaisable, la capacité de la coupe minimale est strictement inférieure à la demande totale :

$$|W_S| < \sum_{l \in S} D_l(t) \quad (48)$$

Cela signifie physiquement que le nombre de travailleurs disponibles maîtrisant au moins une compétence de S (noté $|W_S|$) est insuffisant pour couvrir la demande cumulée en compétences de S demandée par les tâches actives à l'instant t .

3.4.2 Formulation de la coupe injectée

Pour interdire cette situation dans le Problème Maître CP, nous injectons une contrainte cumulative sous forme d'une fonction de pulse :

$$\sum_{i \in \mathcal{A}} \sum_{l \in S} \text{pulse}(act_i, a_{i,l}) \leq |W_S| \quad (49)$$

Cette coupe stipule que, pour tout instant du temps, la somme des demandes en compétences appartenant à S pour toutes les activités s'exécutant simultanément ne doit pas dépasser le nombre de travailleurs qualifiés $|W_S|$. Cette contrainte s'exprime très efficacement en CP grâce à la structure native de fonctions cumulatives et de propagateurs temporels, forçant le Problème Maître à décaler dans le temps certaines activités en conflit pour sa prochaine tentative de résolution.

4 Évaluation Expérimentale

Dans ce chapitre, nous présentons l'évaluation expérimentale des trois approches modélisées : le modèle de programmation par contraintes monolithique classique, le modèle avec bris de symétrie, et le framework de décomposition de Benders basée sur la logique (LBBD). L'objectif est d'analyser leurs performances respectives en termes de temps de résolution, de capacité de passage à l'échelle, et de qualité de convergence face à la flexibilité d'affectation.

4.1 Protocole expérimental et instances de test

4.1.1 Environnement de test

Les expérimentations ont été menées sur une machine équipée d'un processeur Intel Xeon W6-3423 cadencé à 2.1 GHz et de 62 Go de mémoire RAM, sous le système d'exploitation Ubuntu Linux. Les solveurs utilisés sont :

- **IBM ILOG CP Optimizer** (version 22.1) pour la résolution du modèle CP monolithique et du Problème Maître CP dans le cadre de la décomposition.
- L'implémentation de l'algorithme de flot maximum d'Edmonds-Karp du module **NetworkX** pour résoudre le sous-problème d'affectation de Benders à chaque période.

Pour chaque instance de test, la limite de temps de calcul (Time Limit) a été fixée à **500 secondes**.

4.1.2 Description du jeu de données

L'évaluation repose sur un ensemble de **215 instances de test** de référence issues de la bibliothèque d'instances publique **MSPSP-InstLib**¹ introduite par Young et al. [7]. Ce jeu d'instances, regroupé sous le nom de « Set 1a », a été conçu à l'aide d'un générateur d'instances dérivé des travaux d'Almeida et al. [4] et converti au format DataZinc (.dzn) pour être résolu par des modèles MiniZinc ou des solveurs compatibles. Ces instances sont caractérisées par :

- Un nombre d'activités fixe $n = 20$.
- Un nombre de travailleurs $m \in \{10, 13, 15\}$ constituant la ressource multi-compétences.
- Un facteur de complexité réseau ($nc \in \{1.5, 1.8, 2.1\}$), qui régit le nombre de contraintes de précédence et la densité du graphe de projet.
- Un facteur de compétences ($sf \in \{0, 0.5, 0.75, 1.0\}$), caractérisant la diversité et le degré de maîtrise des compétences par les travailleurs du projet.

Ce jeu d'instances offre une diversité de configurations de contraintes (plus ou moins denses) et de ressources (plus ou moins saturées) propice à une analyse comparative rigoureuse.

4.2 Comparaison globale des approches

Les résultats synthétiques obtenus sur l'ensemble des 215 instances pour les trois méthodes de résolution (le modèle CP Classique, le modèle CP avec bris de symétrie et la Décomposition de Benders) sont résumés dans le tableau 1.

¹<https://github.com/youngkd/MSPSP-InstLib>

Table 1: Synthèse globale des résultats de résolution (Time Limit = 500s)

Approche	Inst. résolues	Succès (%)	Temps moy. (s)	Temps hors TO (s)	Gap moy. (%)
CP Classique (Monolithique)	169 / 215	78.6 %	151.31	56.38	2.90 %
CP avec bris de symétrie	164 / 215	76.3 %	159.39	53.46	3.16 %
Benders (LBBD)	209 / 215	97.2 %	24.70	11.05	0.00 %

4.2.1 Analyse des taux de résolution et des temps de calcul

La Décomposition de Benders basée sur la Logique (LBBD) affiche une supériorité nette par rapport aux approches monolithiques. Elle parvient à résoudre à l’optimalité prouvée **97.2 %** du jeu de données (209 instances sur 215), ne subissant que 6 dépassements de limite de temps (timeouts). En comparaison, le modèle CP classique échoue à prouver l’optimalité sur 46 instances (soit 21.4 % de timeouts) et le modèle avec bris de symétrie sur 51 instances (23.7 % de timeouts).

En termes de temps de calcul, l’algorithme LBBD résout les instances en un temps moyen de **24.70 secondes** (dont un temps moyen de seulement **11.05 secondes** pour les instances résolues avant la limite). C’est le modèle le plus performant et le plus rapide globalement.

4.2.2 Comparaison directe sur les instances faciles (communes)

Pour éliminer le biais introduit par les instances où le solveur atteint la limite de temps, nous avons isolé le sous-ensemble des **155 instances résolues à l’optimalité par les trois méthodes**. Les temps moyens de calcul sur ce sous-ensemble commun sont les suivants :

- Temps moyen Benders (LBBD) : **4.55 s**
- Temps moyen CP avec bris de symétrie : **49.48 s**
- Temps moyen CP Classique : **47.00 s**

Sur ces instances communes, la décomposition de Benders est en moyenne **10 fois plus rapide** que le CP classique.

4.3 Analyse fine des performances et discussion

4.3.1 Le phénomène de résolution instantanée par Benders

L’examen détaillé des données révèle un comportement remarquable : sur de nombreuses instances où les modèles CP monolithiques saturent pendant 500 secondes, le framework LBBD converge vers l’optimalité en une fraction de seconde. Par exemple, pour l’instance `inst_set1a_sf0.5_nc1.5_n20_m1` le modèle CP classique expire après 500 secondes en conservant un écart (gap) de 5.0 %, tandis que la décomposition de Benders trouve la solution optimale et en prouve l’optimalité en seulement **0.18 seconde** et en seulement **3 itérations**.

Ce comportement s’explique par la nature des coupes logiques réinjectées. L’algorithme de flot maximum identifie la source minimale de l’infaisabilité spatiale (le sous-ensemble de compétences saturées et le goulot d’étranglement des travailleurs associés). La coupe cumulative réinjectée sous forme de contrainte dans le Problème Maître CP est extrêmement forte : elle élimine instantanément des pans entiers de plannings irréalisables, forçant le maître à décaler les tâches conflictuelles de manière optimale en très peu d’itérations.

4.3.2 Analyse de l’impact des contraintes de bris de symétrie

Un résultat contre-intuitif réside dans les moins bonnes performances du modèle CP enrichi de contraintes de bris de symétrie classiques (non-rotation) par rapport au modèle classique (164

résolutions contre 169). L'expression de ces contraintes de symétrie dans le formalisme CP Optimizer (qui utilise déjà des heuristiques internes très optimisées) introduit une surcharge de calcul (*overhead*) pour les algorithmes de filtrage à chaque nœud de l'arbre qui n'est pas compensée par la réduction de l'arbre de recherche. Le modèle avec bris de symétrie provoque ainsi 5 timeouts supplémentaires par rapport au modèle classique (Figure 3).

4.4 Visualisations graphiques

Les figures 2 et 3 illustrent graphiquement la comparaison des performances à l'aide de profils de performance cumulatifs.

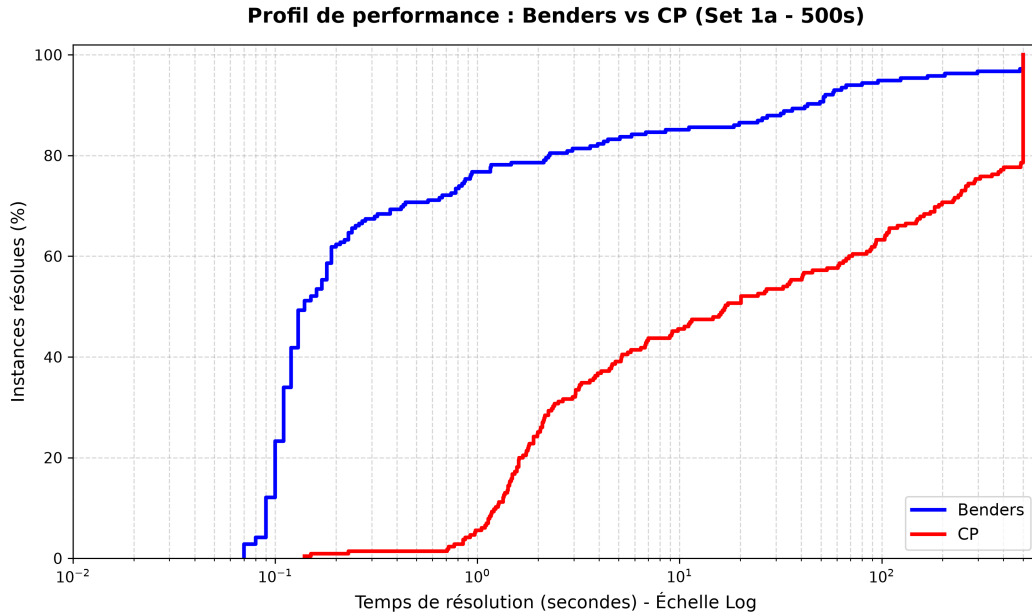


Figure 2: Profil de performance cumulatif : Comparaison du taux de résolution (%) en fonction du temps de calcul (échelle logarithmique) entre la Décomposition de Benders (LBBD) et le CP Classique.

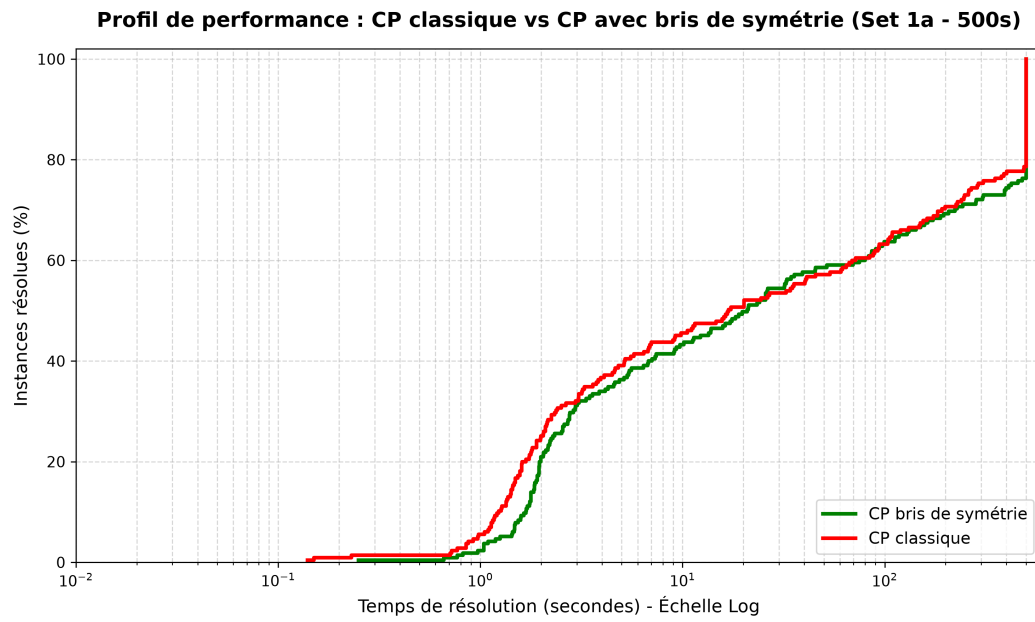


Figure 3: Profil de performance cumulatif : Comparaison du taux de résolution (%) en fonction du temps de calcul (échelle logarithmique) entre le CP Classique et le CP avec bris de symétrie.

Conclusion et Perspectives

References

- [1] O. Bellenguez-Morineau and E. Néron, “A branch-and-bound method for solving multi-skill project scheduling problem,” *RAIRO-Operations Research*, vol. 41, no. 2, pp. 155–170, 2007.
- [2] ———, “Multi-mode and multi-skill project scheduling problem,” in *Resource-Constrained Project Scheduling: Models, Algorithms, Extensions and Applications*, C. Artigues, S. Demassey, and E. Néron, Eds. Wiley Online Library, 2008, pp. 149–160.
- [3] J. Snauwaert and M. Vanhoucke, “A classification and new benchmark instances for the multi-skilled resource-constrained project scheduling problem,” *European Journal of Operational Research*, vol. 307, no. 1, pp. 1–19, 2023.
- [4] B. F. Almeida, I. Correia, and F. Saldanha-da-Gama, “Modeling frameworks for the multi-skill resource-constrained project scheduling problem: A theoretical and empirical comparison,” *International Transactions in Operational Research*, vol. 26, no. 3, pp. 946–967, 2019.
- [5] I. Correia and F. Saldanha-da-Gama, “A modeling framework for project staffing and scheduling problems,” in *Handbook on Project Management and Scheduling Vol. 1*, C. Schwindt and J. Zimmermann, Eds. Springer, 2015, pp. 547–564.
- [6] C. Montoya, O. Bellenguez-Morineau, E. Pinson, and D. Rivreau, “Branch-and-price approach for the multi-skill project scheduling problem,” *Optimization Letters*, vol. 8, pp. 1721–1734, 2014.
- [7] K. D. Young, T. Feydy, and A. Schutt, “Constraint programming applied to the multi-skill project scheduling problem,” in *Principles and Practice of Constraint Programming: 23rd International Conference (CP 2017), Melbourne, VIC, Australia, August 28–September 1, 2017, Proceedings 23*. Springer, 2017, pp. 308–317.
- [8] O. Polo-Mejía, C. Artigues, P. Lopez, and V. Basini, “Mixed-integer/linear and constraint programming approaches for activity scheduling in a nuclear research facility,” *International Journal of Production Research*, vol. 58, no. 23, pp. 7149–7166, 2020.
- [9] J. N. Hooker, *Logic-based methods for optimization: Combining optimization and constraint satisfaction*. New York: John Wiley and Sons, 2000.
- [10] ———, “Logic-based benders decomposition for large-scale optimization,” *Large scale optimization in supply chains and smart manufacturing: Theory and applications*, pp. 1–26, 2019.
- [11] A. A. Cire, E. Coban, and J. N. Hooker, “Logic-based benders decomposition for planning and scheduling: A computational analysis,” *The Knowledge Engineering Review*, vol. 31, no. 5, pp. 440–451, 2016.
- [12] W. Yi-fan, S. Fu-quan, L. Shi-xin, and C. Di, “A new method to solve project scheduling problems with multi-skilled workforce constraints,” in *2013 25th Chinese Control and Decision Conference (CCDC)*. IEEE, 2013, pp. 2408–2412.
- [13] C. Juvin, C. Artigues, P. Lopez, and R. Brux, “Logic-based benders method for multi-skill resource-constrained project scheduling with allocation and skill flexibility,” *Preprint submitted to Elsevier*, 2026.
- [14] R. M. Karp, *Reducibility among combinatorial problems*. Springer, 2010.
- [15] C. H. Papadimitriou and K. Steiglitz, *Combinatorial optimization: Algorithms and complexity*. Courier Corporation, 1998.

- [16] E. L. Demeulemeester and W. S. Herroelen, *Project scheduling: A research handbook*. Springer Science & Business Media, 2006, vol. 49.
- [17] H. Li and K. Womer, “Scheduling projects with multi-skilled personnel by a hybrid MILP/CP Benders decomposition algorithm,” *Journal of Scheduling*, vol. 12, no. 3, pp. 281–298, 2009.
- [18] H. Li, “Benders decomposition approach for project scheduling with multi-purpose resources,” in *Handbook on Project Management and Scheduling Vol. 1*, C. Schwindt and J. Zimmermann, Eds. Springer, 2015, pp. 587–601.
- [19] O. Polo-Mejía, C. Artigues, P. Lopez, L. Mönch, and V. Basini, “Heuristic and metaheuristic methods for the multi-skill project scheduling problem with partial preemption,” *International Transactions in Operational Research*, vol. 30, no. 2, pp. 858–891, 2023.
- [20] O. Polo Mejía, “Operational research approach for optimising the operations of a nuclear research laboratory,” Ph.D. dissertation, Institut National des Sciences Appliquées de Toulouse, Sep 2019. [Online]. Available: <https://hal.science/tel-02309284>
- [21] H. Fahimi and C.-G. Quimper, “Linear-time filtering algorithms for the disjunctive constraint,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28(1), 2014.
- [22] P. Vilím, “Edge finding filtering algorithm for discrete cumulative resources in $\mathcal{O}(kn \log n)$,” in *International Conference on Principles and Practice of Constraint Programming (CP)*. Springer, 2009, pp. 802–816.
- [23] Y. Ouellet and C.-G. Quimper, “A $O(n \log^2 n)$ checker and $O(n^2 \log n)$ filtering algorithm for the energetic reasoning,” in *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*. Springer, 2018, pp. 477–494.
- [24] J. Carlier and E. Néron, “Computing redundant resources for the resource constrained project scheduling problem,” *European Journal of Operational Research*, vol. 176, no. 3, pp. 1452–1463, 2007.
- [25] P. Baptiste and N. Bonifas, “Redundant cumulative constraints to compute preemptive bounds,” *Discrete Applied Mathematics*, vol. 234, pp. 168–177, 2018.
- [26] J. Benders, “Partitioning procedures for solving mixed-variables programming problems,” *Numerische Mathematik*, vol. 4, no. 1, pp. 238–252, 1962.
- [27] A. Nasirian, L. Zhang, A. M. Costa, and B. Abbasi, “Multiskilled workforce staffing and scheduling: A logic-based Benders’ decomposition approach,” *European Journal of Operational Research*, vol. 323, no. 1, pp. 20–33, 2025.
- [28] M. Lombardi and M. Milano, “A min-flow algorithm for minimal critical set detection in resource constrained project scheduling,” *Artificial Intelligence*, vol. 182, pp. 58–67, 2012.
- [29] F. Stork and M. Uetz, “On the generation of circuits and minimal forbidden sets,” *Mathematical programming*, vol. 102, pp. 185–203, 2005.